Experiment 2: Product Listing Page

ProductCard.js

```
import React from "react";
function ProductCard({name, price, category }) {
  return (
    <div style={{border: "1px solid black", margin: "10px", padding: "10px" }}>
      <h3>{name}</h3>
      <p>Price: ₹{price}</p>
      <p>Category: {category}</p>
      <button onClick={() => alert(`${name} Purchased`)}>
        Buy Now
      </button>
    </div>
  );
}

export default ProductCard;
```

App.js

```
import React from "react";
import ProductCard from "./ProductCard";

function App() {
  const products = [
    { id: 1, name: "Laptop", price: 50000, category: "Electronics" },
    { id: 2, name: "Mobile", price: 20000, category: "Electronics" },
    { id: 3, name: "Shoes", price: 1500, category: "Fashion" }
  ];

  return (
    <div>
      <h1>Product Listing</h1>

      {products.map((product) => (
        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
          category={product.category}
        />
      ))}
    </div>
  );
}
export default App;
```

Experiment No: 2

Title

Product Listing Page Using React Components

Aim

1. **To create reusable React components.**
2. **To display product information using props.**
3. **To understand component-based architecture in React.**
4. **To implement event handling using buttons.**

Software Requirements

- **Windows 10/11**
- **Node.js**
- **React.js**
- **Visual Studio Code**
- **Google Chrome**

Theory

React applications are built using components, which are reusable blocks of code. Components help organize the user interface and improve code reusability.

Props (Properties) are used to pass data from a parent component to a child component. In this experiment, product details such as name, price, and category are passed from the App component to the ProductCard component using props.

The map() function is used to display multiple products dynamically from an array. When the Buy Now button is clicked, an alert message is displayed.

Algorithm

1. **Create a React application.**
2. **Create a ProductCard component.**
3. **Store product details in an array.**
4. **Pass product data as props to the ProductCard component.**
5. **Use the map() function to display all products.**
6. **Add a Buy Now button.**

7. Display an alert message when the button is clicked.

Execution Steps

Step 1

Open Command Prompt.

Step 2

Create a React application.

```
npx create-react-app product-list
```

Step 3

Move to the project folder.

```
cd product-list
```

Step 4

Open the project in Visual Studio Code.

```
code .
```

Step 5

Inside the src folder, create a new file named:

ProductCard.js

Step 6

Copy the given ProductCard.js code into the file.

Step 7

Replace the contents of App.js with the given program.

Step 8

Save all files.

Step 9

Run the application.

```
npm start
```

Step 10

Open the browser.

http://localhost:3000

Step 11

Verify that all product cards are displayed.

Step 12

Click the Buy Now button and observe the alert message.

Folder Structure

product-list

```
|
├── src
|    ├── App.js
|    ├── ProductCard.js
|    ├── index.js
|   └── App.css
|
├── public
├── package.json
└── node_modules
```

Program

ProductCard.js

(ProductCard.js code from the experiment.)

App.js

(App.js code from the experiment.)

Sample Input

Product 1

Name : Laptop

Price : ₹50000

Category : Electronics

Product 2

Name : Mobile

Price : ₹20000

Category : Electronics

Product 3

Name : Shoes

Price : ₹1500

Category : Fashion

Expected Output

Product Listing

Laptop

Price : ₹50000

Category : Electronics

[Buy Now]

Mobile

Price : ₹20000

Category : Electronics

[Buy Now]

Shoes

Price : ₹1500

Category : Fashion

[Buy Now]

When Buy Now is clicked:

Laptop Purchased

(or the corresponding product name)

Result

Thus, the Product Listing Page using React Components and Props was developed and executed successfully.